DOMAINFORGE

Product Requirements Document

A full-stack domain search, management, and ordering platform designed as a portfolio-grade engineering project.

Document field	Details
Version	1.0
Status	Draft for implementation
Prepared	29 October 2026
Product type	Responsive web application + REST API
Primary audience	Recruiters and engineering reviewers; prospective users of a domain platform
Initial implementation stack	React, JavaScript, Node.js, Express, PostgreSQL

Product principle
Make domain-related tasks clear, trustworthy, secure, and easy to understand. Never imply that a domain is available, purchased, or configured unless the relevant external provider has confirmed the result.

Prepared for guided implementation and portfolio demonstration. This document defines intended behaviour; it does not claim that any feature has already been implemented or deployed.

Document control

Item	Definition
Purpose	Define product scope, requirements, user experience, data, APIs, security, quality, and acceptance criteria before implementation.
Primary success outcome	A demonstrable, well-tested repository that proves practical JavaScript full-stack engineering skills relevant to domain services.
MVP boundary	A portfolio application with truthful simulated domain operations unless a real registrar API is integrated and verified.
Audience	Developer, technical reviewer, recruiter, and test users.
Review rule	Any change to scope, payment behaviour, provider integration, or security requirements must be reflected in this document or a linked decision record.

Table of contents

- 1. Executive summary
- 2. Problem statement and opportunity
- 3. Product vision, goals, and non-goals
- 4. Users and user stories
- 5. Scope and release strategy
- 6. Functional requirements
- 7. User journeys and screen requirements
- 8. Domain search rules and provider integration
- 9. Technical architecture
- 10. Data model
- 11. API specification
- 12. Security, privacy, and abuse prevention
- 13. Accessibility, usability, and responsive design
- 14. Observability, reliability, and performance
- 15. Testing and quality strategy
- 16. Analytics and success metrics
- 17. Deployment and operations
- 18. Acceptance criteria
- 19. Risks, assumptions, and dependencies
- 20. Milestones and delivery plan
- 21. GitHub repository and documentation requirements
- 22. Open decisions and glossary

1. Executive summary

DomainForge is a responsive web application that demonstrates the core workflow of a domain registrar: searching for domain names, reviewing results, creating an account, managing saved domain records, and moving through an order/checkout experience. The project is intended to serve as a credible software-engineering portfolio piece, with particular relevance to a role requiring JavaScript, Node.js, HTTP APIs, validation, security, testing, debugging, and clear documentation.

The first release will prioritize a reliable end-to-end vertical slice over a broad set of incomplete features. Domain syntax validation and mock search results may be used in local development, but the interface and API must label simulated data clearly. Live availability, registration, DNS changes, and payment completion require verified external integrations and must never be fabricated.

The application will use React on the frontend, Node.js and Express for the REST API, PostgreSQL for persistent data, and JavaScript throughout. The architecture will separate routing, controllers, services, validation, and persistence so each part can be tested and maintained independently.

Primary deliverable: a public GitHub repository with meaningful commit history, automated tests, a clear README, architecture notes, screenshots, and—if feasible—a deployed demo with no real customer data or live financial transactions.

2. Problem statement and opportunity

2.1 Problem

Individuals and small businesses often find domain-related workflows confusing: choosing a name, understanding extension differences, determining whether a name is truly available, comparing costs, and managing domain settings. A domain platform must make these operations understandable while protecting user accounts and avoiding misleading claims.

2.2 Opportunity

DomainForge provides a practical way to demonstrate the engineering behind these workflows. The portfolio value comes from correct state handling, explicit provider boundaries, secure API design, clear error behaviour, tests, and documented trade-offs—not from pretending to operate a real registrar.

2.3 User problem statements

- As a prospective customer, I need to search a domain name and understand whether the result is verified or simulated.
- As a customer, I need to sign in and see only the domain records associated with my account.
- As a customer, I need clear status, pricing, and order information before I take an action.
- As an engineer or reviewer, I need predictable API responses and tests that make defects easier to find.
- As an administrator/developer, I need to identify provider failures and diagnose issues without exposing secrets or personal data.

3. Product vision, goals, and non-goals

3.1 Vision

Build a trustworthy, understandable domain-management experience and demonstrate production-minded full-stack engineering through a small but complete application.

3.2 Goals

- Deliver an accessible, mobile-responsive frontend.
- Provide a documented REST API with validation and consistent error responses.
- Persist user-owned domain records and orders in PostgreSQL.
- Demonstrate secure authentication, authorization, and ownership checks.
- Include automated unit, API integration, and critical end-to-end tests.

- Make the difference between simulated and provider-confirmed operations obvious.
- Publish clear setup, testing, architecture, and limitation documentation.

3.3 Non-goals for MVP

- Operating as a legally accredited registrar or reseller.
- Actually registering, renewing, transferring, or deleting live domains without an approved provider integration.
- Making real charges or storing card details.
- Providing a production-grade DNS hosting service, DNSSEC management, email hosting, or website hosting.
- Collecting government IDs, NIN, BVN, or other high-risk identity data.
- Supporting multi-vendor marketplace features, payroll, accounting, or unrelated commerce modules.

Scope rule: features not listed in the MVP must not delay completion of the core domain-search-to-demo-order journey.

4. Users and user stories

Persona	Needs	MVP priority
Visitor / prospective customer	Search a name, understand results and indicative prices, learn limitations.	P0
Registered customer	Sign in, view their domain records, inspect order history, manage demo records.	P0
Engineering reviewer	Run the app locally, inspect architecture, execute tests, understand API behaviour.	P0
Maintainer / developer	Configure environment, troubleshoot errors, rotate secrets, inspect logs.	P0
Real registrar customer	Obtain live availability, register and manage real domains.	Out of MVP unless provider integration is completed

4.1 User stories

- **US-01:** As a visitor, I can enter a domain name so I can check whether the input is valid.
- **US-02:** As a visitor, I can see supported extensions and indicative prices, with mock results clearly labelled.
- **US-03:** As a visitor, I receive useful guidance when a name is invalid, unsupported, or a search provider fails.
- **US-04:** As a customer, I can create an account and sign in securely.
- **US-05:** As a customer, I can see only records and orders that belong to my account.
- **US-06:** As a customer, I can add a mock-available result to a demo order.
- **US-07:** As a customer, I can see the status of a demo order and understand that no real registration or payment occurred.
- **US-08:** As a customer, I can inspect a domain record and edit permitted demo settings.
- **US-09:** As a reviewer, I can run automated tests and follow setup instructions from a clean checkout.
- **US-10:** As a maintainer, I can diagnose failures through structured logs without logging passwords, tokens, or payment secrets.

5. Scope and release strategy

5.1 Priority definitions

Priority	Meaning
P0 — Required	Needed for the core portfolio demonstration and MVP acceptance.
P1 — Important	Improves realism and quality after the core journey works.
P2 — Optional	Only implement if time remains and P0/P1 quality is stable.

5.2 MVP scope

Capability	Priority	MVP behaviour
Public landing/search page	P0	Responsive domain-search form and clear demo disclaimer.
Domain input validation	P0	Client and server validation; server is authoritative.
Search results	P0	Mock provider returns clearly labelled simulated results.
Account registration/login/logout	P0	Secure password hashing and session-based authentication.
Customer dashboard	P0	List only the signed-in user's records and orders.
Demo order flow	P0	Create order with an explicit simulated status; no real charge.
Domain detail/settings	P0	Edit only permitted demo fields; no real DNS changes.
PostgreSQL persistence	P0	Migrations and constraints for core entities.
Automated tests	P0	Validation, authorization, API, and core-flow coverage.
README and API documentation	P0	Reproducible local setup and honest feature limitations.
Live registrar adapter	P1	Integrate only if credentials, terms, and sandbox are available.
Payment sandbox	P1	Optional test-mode provider integration; no live payment capture.
Admin analytics dashboard	P2	Not needed for MVP.
Email verification / password reset	P1	Add if a safe email service is available; otherwise document limitation.

6. Functional requirements

6.1 Landing page and navigation

- FR-001 (P0): Show product name, concise value proposition, domain search form, and navigation to login/register/dashboard.
- FR-002 (P0): Explain that this is a portfolio demonstration and that mock results do not establish real domain availability.
- FR-003 (P0): Support keyboard navigation, visible focus states, and meaningful labels.
- FR-004 (P1): Show a short explanation of domain extensions and sample pricing assumptions.

6.2 Domain input and validation

- FR-010 (P0): Trim surrounding whitespace and normalize domain input consistently.
- FR-011 (P0): Validate length and permitted label syntax on the server; reject empty values, invalid separators, malformed labels, and unsupported inputs.
- FR-012 (P0): Never rely on frontend validation alone.
- FR-013 (P0): Return field-level, user-safe validation messages without stack traces.
- FR-014 (P1): Handle internationalized domain names explicitly; either normalize using a well-tested library or clearly state the MVP limitation.

- FR-015 (P0): Keep syntax validity separate from availability status.

6.3 Domain search

- FR-020 (P0): Accept a domain query and supported extension set.
- FR-021 (P0): Return a structured result for each requested extension.
- FR-022 (P0): In mock mode, mark the source as 'simulation' and availability as 'unknown' or a clearly defined demo state; never imply live verification.
- FR-023 (P0): Display loading, success, empty, invalid-input, and provider-error states.
- FR-024 (P0): Disable or safely de-duplicate repeated submissions while a search is pending.
- FR-025 (P1): If a real provider is added, show provider-confirmed availability with a timestamp and handle stale results.
- FR-026 (P1): Add rate limiting and basic abuse controls to the public search endpoint.

6.4 Authentication and account

- FR-030 (P0): Support registration, login, and logout.
- FR-031 (P0): Hash passwords using a suitable password-hashing algorithm and vetted library; never store plaintext passwords.
- FR-032 (P0): Use secure, HttpOnly cookies for sessions; configure SameSite and Secure attributes appropriately for deployment.
- FR-033 (P0): Protect state-changing actions against CSRF when cookie-based authentication is used.
- FR-034 (P0): Return generic login failures that do not reveal whether an email address exists.
- FR-035 (P0): Apply authentication and authorization middleware to protected routes.
- FR-036 (P1): Implement email verification and password reset only when a reliable delivery mechanism is configured.
- FR-037 (P0): Avoid collecting unnecessary personal data.

6.5 Dashboard and domain records

- FR-040 (P0): Show a summary of the signed-in user's demo domain records and orders.
- FR-041 (P0): Ensure each record query is scoped to the authenticated user's ID.
- FR-042 (P0): Show domain, demo status, created date, and relevant expiry/renewal fields when applicable.
- FR-043 (P0): Provide a clear empty state and a route back to domain search.
- FR-044 (P0): Permit editing only documented demo fields; reject changes to immutable ownership/provider identifiers.
- FR-045 (P0): Never allow the browser to choose or override the owner ID for a protected operation.

6.6 Orders and checkout

- FR-050 (P0): Create an order from a supported demo search result.
- FR-051 (P0): Store a server-calculated price snapshot and currency; do not trust a client-supplied total.
- FR-052 (P0): Use explicit order states, such as draft, pending_demo, completed_demo, cancelled, and failed.
- FR-053 (P0): Display a clear notice that demo checkout does not charge money or register a domain.
- FR-054 (P0): Prevent duplicate submission from creating duplicate demo orders where feasible using idempotency keys or safe state checks.
- FR-055 (P1): If a payment sandbox is integrated, verify payment status server-side and verify webhook signatures; never trust a frontend success redirect alone.
- FR-056 (P0): Do not store card numbers, CVVs, or payment-provider secrets.

6.7 Error handling and supportability

- FR-060 (P0): Use a consistent API error shape containing a stable code, safe message, and optional field details.
- FR-061 (P0): Return appropriate HTTP status codes.

- FR-062 (P0): Log unexpected server errors with a request/correlation ID while redacting sensitive data.
- FR-063 (P0): Never send stack traces, SQL details, secrets, or internal file paths to clients in production mode.
- FR-064 (P1): Add health/readiness endpoints that do not reveal credentials or internal configuration.

7. User journeys and screen requirements

7.1 Visitor searches for a domain

1. Visitor opens the landing page.
2. Visitor enters a domain query and submits it.
3. Frontend performs immediate basic checks and sends the request to the API.
4. API validates the input and calls the configured search provider adapter.
5. The adapter returns a normalized result, with the source identified as simulation or live provider.
6. Frontend shows the result, price assumptions, and the next available action.
7. If the provider fails, the UI explains that the search could not be completed and offers a retry.

7.2 Customer creates a demo order

1. Customer signs in or registers.
2. Customer selects a supported result.
3. Frontend asks the API to create an order; it does not submit a trusted total.
4. API revalidates the requested domain, checks the server-side price configuration, and creates an order for the authenticated user.
5. Checkout screen explains that no real payment or registration occurs.
6. Customer confirms the demo action.
7. API transitions the order to the appropriate demo state and creates a demo domain record only according to the documented state model.
8. Dashboard displays the updated order and domain record.

7.3 Screen inventory

Screen	Required elements	Important states
Landing/search	Brand, search form, extension choices, demo notice, navigation.	Idle, loading, invalid input, results, no result, error.
Search results	Domain, source, availability semantics, indicative price, next action.	Simulation vs live result, stale result, provider error.
Register/login	Labeled fields, validation, submit state, privacy note.	Invalid, pending, success, generic authentication error.
Dashboard	Domain summary, order summary, recent activity, search shortcut.	Loading, populated, empty, API error.
Domain detail	Domain identity, demo/live status, supported settings, audit timestamps.	Read-only, edit, validation failure, save success.
Order detail	Items, price snapshot, currency, state, demo/payment disclaimer.	Pending, completed demo, cancelled, failed.
Not found/error	Helpful message and recovery navigation.	404, unavailable service, unauthorized.

7.4 UX principles

- Make the main action—searching a domain—obvious.
- Use plain language; explain domain availability and demo limitations near the result/action.

- Avoid false urgency, deceptive pricing, and fake success states.
- Use clear success/error feedback and preserve user input when safe to do so.
- Use consistent spacing, typography, button states, and form error presentation.
- Do not depend on colour alone to communicate status.

8. Domain search rules and provider integration

8.1 Provider abstraction

Implement a DomainProvider interface or equivalent service boundary. The initial MockDomainProvider will return deterministic, clearly labelled demonstration responses. A future RegistrarDomainProvider can implement the same contract without making controllers dependent on a specific vendor.

8.2 Normalized result contract

Field	Meaning
domain	Canonical domain string requested or evaluated.
isValid	Whether syntax validation passed.
availability	One of unknown, available, unavailable, or error; only a trusted provider may assert live availability.
source	simulation or provider identifier.
checkedAt	Timestamp for the result, when applicable.
price	Optional server-configured indicative price and currency.
message	Safe user-facing explanation, not provider secrets or raw errors.

8.3 Rules

- Do not use a DNS lookup alone to claim a domain is available for registration.
- Treat RDAP/WHOIS, DNS, and registrar availability as different signals; the registrar or registry workflow is authoritative for a purchase.
- Use provider timeouts, bounded retries where safe, and normalized error handling.
- Do not retry non-idempotent registration/payment operations blindly.
- Cache only when provider terms and freshness requirements allow it; label cached data and timestamps.
- Do not scrape a registrar or violate provider terms to simulate an integration.

9. Technical architecture

9.1 Components

Component	Responsibility
React client	Presentation, form state, route navigation, API calls, accessibility.
Express application	HTTP configuration, middleware, route registration, error boundary.
Routes/controllers	Map HTTP requests to application services and safe response formats.
Services	Business rules for domain search, accounts, orders, and domain records.
Validators	Runtime validation of query, path, and body inputs.
Repositories/ORM	Persistence and database queries with scoped ownership checks.
PostgreSQL	Durable relational storage, constraints, and transaction support.
Provider adapter	Mock domain search initially; optional real registrar/payment provider later.
Test suite	Unit, API integration, and browser-level verification.

9.2 Backend structure

Recommended structure as the application grows:

```
server/
  src/
    app.js
    server.js
    config/
    routes/
    controllers/
    services/
    repositories/
    validators/
    middleware/
    providers/
    utils/
  tests/
```

9.3 Architectural rules

- Keep HTTP-specific logic out of business services where practical.
- Validate external input at the boundary and enforce business rules in the service layer.
- Use database transactions for multi-step state changes that must be atomic.
- Use explicit migrations; do not rely on undocumented manual database edits.
- Keep secrets in environment configuration or a secret manager, never in source control.
- Avoid overengineering: introduce abstractions when they support a real requirement or testability.

10. Data model

The initial relational model is intentionally small. Names and columns can evolve during implementation, but ownership, uniqueness, and state transitions must remain explicit.

Entity	Key fields / constraints
User	id (UUID), email (normalized and unique), password_hash, created_at, updated_at, status.
Session	id or token hash, user_id FK, expires_at, created_at; store only the minimum session data required.
DomainRecord	id UUID, user_id FK, fqdn, source_mode, status, created_at, updated_at, expires_at nullable; unique constraints as appropriate for demo ownership.

Entity	Key fields / constraints
Order	id UUID, user_id FK, order_number unique, status, currency, subtotal_minor_units, total_minor_units, created_at, updated_at, idempotency_key nullable.
OrderItem	id UUID, order_id FK, fqdn, term, unit_price_minor_units, currency, item_status; price snapshot immutable after order creation.
ProviderEvent (optional)	id, provider, external_event_id unique, event_type, processed_at, safe metadata; used for webhook idempotency.
AuditEvent (optional/P1)	id, actor_user_id nullable, action, target_type, target_id, timestamp, safe metadata; never store credentials or sensitive payloads.

10.1 Data rules

- Use UUIDs or another non-sequential public identifier for exposed resource IDs.
- Store money as integer minor units (for example, kobo) plus an explicit currency code; avoid floating-point totals.
- Use UTC timestamps in storage and format them for the user's locale in the UI.
- Apply foreign keys, uniqueness constraints, and suitable indexes.
- Normalize email addresses consistently before uniqueness checks.
- Define deletion/retention behaviour before adding real customer data.
- Use fictional seed data only; never commit real customer records.

10.2 Domain status semantics

Suggested statuses include demo_pending, demo_active, provider_pending, active, expired, failed, and cancelled. Only statuses justified by actual system evidence may be used. For MVP, use explicit demo-prefixed statuses and do not mix them with live registration statuses.

11. API specification

Base path: /api/v1. The initial health endpoint may remain at /api/health. JSON is the default request/response format. All endpoints must return appropriate HTTP status codes and a consistent error envelope.

11.1 Endpoint inventory

Method + path	Auth	Purpose
GET /api/health	Public	Liveness check; no secrets or infrastructure details.
GET /api/v1/domains/search?q=...	Public + rate limited	Validate query and return normalized search results.
POST /api/v1/auth/register	Public + rate limited	Create account after validation.
POST /api/v1/auth/login	Public + rate limited	Authenticate and establish session.
POST /api/v1/auth/logout	Authenticated	Invalidate session.
GET /api/v1/auth/me	Authenticated	Return current user's safe profile fields.
GET /api/v1/domains	Authenticated	List current user's domain records.
GET /api/v1/domains/:id	Authenticated + owner check	Retrieve a domain record belonging to the user.
PATCH /api/v1/domains/:id	Authenticated + owner check	Update explicitly allowed demo fields.
POST /api/v1/orders	Authenticated	Create order from server-validated items and price configuration.
GET /api/v1/orders	Authenticated	List current user's orders.
GET /api/v1/orders/:id	Authenticated + owner check	Retrieve a user's order.
POST /api/v1/orders/:id/confirm-demo	Authenticated + owner check	Confirm a demo order; never charges money.

11.2 Example search response

```
{
  "data": {
    "query": "example",
    "source": "simulation",
    "results": [
      {
        "domain": "example.com",
        "isValid": true,
        "availability": "unknown",
        "price": { "amountMinor": 1500000, "currency": "NGN" },
        "message": "Demonstration result; availability is not verified."
      }
    ]
  }
}
```

The numeric price above is illustrative only and must be replaced with a clearly documented demo value. Confirm currency conventions and whether amountMinor is expressed in kobo before implementing; this PRD assumes 100 kobo = ■1.

11.3 Error response

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please check the domain name and try again.",
    "details": [{ "field": "q", "issue": "Invalid domain input." }],
    "requestId": "correlation-id"
  }
}
```

11.4 HTTP status guidance

- 200: successful read or update with a response body.
- 201: resource created.
- 204: successful operation with no response body.
- 400: malformed request or invalid input where appropriate.
- 401: authentication required or invalid session.

- 403: authenticated user lacks permission, when revealing resource existence is safe.
- 404: resource not found; consider returning 404 for another user's resource to reduce enumeration.
- 409: conflicting state or duplicate resource.
- 422: semantically invalid request if consistently adopted.
- 429: rate limit exceeded.
- 500: unexpected server failure with a generic client message.
- 502/503/504: upstream provider failure, unavailable dependency, or timeout where appropriate.

12. Security, privacy, and abuse prevention

Security is a release requirement, not a final polish task. Use current, maintained libraries and review their documentation before implementation.

12.1 Authentication and session security

- Use a vetted password hashing algorithm such as Argon2id or bcrypt with an appropriate work factor.
- Use cryptographically secure session identifiers and store session state server-side or use a carefully designed, validated alternative.
- Set HttpOnly, Secure (in production), and appropriate SameSite cookie attributes.
- Rotate or regenerate sessions after login and privilege changes; expire sessions.
- Protect cookie-authenticated state-changing requests against CSRF.
- Rate-limit login and registration; avoid account enumeration through distinct error messages.
- Never log passwords, raw session IDs, reset tokens, or authorization headers.

12.2 Authorization and data access

- Apply authorization checks on the server for every protected resource.
- Derive the owner ID from the authenticated session, not from request body or query parameters.
- Test cross-account access attempts for every domain/order endpoint.
- Use least-privilege database credentials and parameterized queries/ORM APIs.
- Validate allowed update fields to prevent mass assignment.

12.3 Web application protections

- Use Helmet and a considered Content Security Policy appropriate to the deployed frontend.
- Configure CORS for known origins; CORS is not authentication and must not be treated as a security boundary.
- Set request body size limits and rate limits.
- Avoid unsafe HTML rendering; encode output and sanitize any supported rich-text input.
- Use secure error handling and avoid leaking stack traces or infrastructure information.
- Keep dependencies updated and run dependency/security checks.

12.4 Payment and provider integration

- Never store card numbers or CVVs.
- Use a payment provider's hosted checkout/tokenization approach if a sandbox is added.
- Verify webhook signatures and event authenticity; make webhook handling idempotent.
- Do not mark an order paid based only on browser redirect parameters.
- Keep provider credentials in environment secrets; use test keys in development.
- Separate demo order states from real payment/registration states.

12.5 Privacy

- Collect only data needed for the demo.
- Do not request NIN, BVN, government ID, or bank information.
- Document what data is stored and how to reset seed/demo data.
- Use fictional accounts and domain names for public screenshots.
- Define a deletion/retention policy before introducing real users or live transactions.

13. Accessibility, usability, and responsive design

13.1 Accessibility requirements

- Aim for WCAG 2.2 AA practices where feasible.
- Use semantic HTML and correctly associated form labels.
- Support keyboard-only navigation and visible focus indicators.
- Ensure text/background contrast is sufficient.
- Announce dynamic search results and validation feedback to assistive technology.
- Use accessible button names and meaningful link text.
- Do not communicate availability or errors through colour alone.

13.2 Responsive requirements

- Support narrow mobile screens, tablets, and desktop layouts without horizontal overflow.
- Keep search and checkout actions usable on touch devices.
- Use responsive tables/cards for dashboard data.
- Test common viewport sizes and browser zoom.
- Avoid unnecessary animation; respect reduced-motion preferences.

14. Observability, reliability, and performance

14.1 Logging

- Emit structured logs with timestamp, level, request ID, route, status code, and duration.
- Redact credentials, passwords, cookies, tokens, and unnecessary personal data.
- Use distinct log levels and avoid logging every successful request noisily in production.
- Provide a documented way to reproduce and investigate common errors.

14.2 Reliability

- Set timeouts for upstream API calls.
- Use bounded retries only for safe/idempotent operations.
- Fail gracefully when the database or provider is unavailable.
- Use transactions for order state changes that update multiple records.
- Do not expose partial success if an operation did not complete atomically.

14.3 Performance targets

Measure	Initial target	Notes
Frontend initial interaction	Aim for a usable page within 3 seconds on a typical mid-range mobile connection.	Validate with a repeatable tool; environment-dependent.
API response (local, non-provider endpoints)	p95 under 300 ms with seeded development data.	Measure; not a guarantee for hosted environments.
Domain provider timeout	Configurable; initial default around 5 seconds.	Provider-specific limits may require adjustment.
Error response	No unhandled promise rejection or uncaught exception for expected invalid inputs.	Unexpected failures must be logged safely.
Automated test suite	Runs reliably from a clean checkout.	Prefer deterministic tests; mock external services.

15. Testing and quality strategy

15.1 Unit tests

- Domain normalization and validation, including edge cases.
- Pricing calculation and integer minor-unit handling.
- Order state transitions.
- Provider adapter normalization and error mapping.
- Input schema validation.

15.2 API integration tests

- Valid and invalid search requests.
- Registration and login validation.
- Protected route access without a session.
- User A cannot read or modify User B's domain/order.
- Order creation ignores client-supplied price totals.
- Duplicate demo confirmation does not duplicate records.
- Provider timeout and unexpected errors return safe responses.
- Rate-limit behaviour for public endpoints.

15.3 End-to-end tests

- Visitor can search and see the demo disclaimer.
- User can register, log in, create a demo order, and view it on the dashboard.
- Invalid input produces accessible feedback.
- Logout removes access to protected screens/endpoints.
- Responsive navigation and primary flow work at mobile and desktop sizes.

15.4 Definition of done

- Acceptance criteria pass.
- Tests are added for meaningful business logic and security boundaries.
- Lint/test commands pass locally and in CI.
- No secrets or private user data are committed.
- Errors are handled and the UI includes loading, empty, and failure states.

- README/API docs are updated.
- Changes are committed with a clear message.

16. Analytics and success metrics

For a portfolio project, metrics are primarily verification indicators rather than business claims. Do not add invasive analytics or collect personal data merely to produce numbers.

Metric	Target / evidence
Core flow completion	A reviewer can search, register, create a demo order, and view it without manual database edits.
Automated test coverage	Aim for strong coverage of business-critical services and authorization paths; do not optimize only for a percentage.
Security checks	Cross-account access tests pass; no secrets committed; dependency checks run.
Documentation quality	A new developer can set up the app using the README alone.
API consistency	All documented endpoints have stable response/error formats and tests.
Accessibility	Keyboard and form checks completed; no known blocking issue in the primary flow.
Deployment	A reviewer can access a demo or follow clear local setup instructions.

17. Deployment and operations

17.1 Environments

- Local: mock domain provider, local PostgreSQL, test-only credentials.
- CI: isolated test database or test fixtures; external providers mocked.
- Demo/staging: hosted application with synthetic data, limited privileges, and no live charges.
- Production: out of scope unless real provider, legal, privacy, support, and operational requirements are addressed.

17.2 Environment variables

Provide `.env.example` with names and safe placeholders only. Likely variables include NODE_ENV, PORT, DATABASE_URL, SESSION_SECRET, CLIENT_URL, and optional provider test credentials. Validate required environment variables at startup and fail with a safe, actionable message if configuration is invalid.

17.3 Database management

- Use versioned schema migrations.
- Document database creation and migration commands.
- Seed only clearly synthetic data.
- Do not use production data in local development.
- Use least-privilege credentials and a separate test database.

17.4 CI pipeline

- Install dependencies using the committed lockfile.
- Run linting/format checks if configured.
- Run unit and API integration tests.
- Build the frontend.
- Run dependency audit/security checks where practical.
- Fail the pipeline if required checks fail.

18. Acceptance criteria

ID	Acceptance criterion	Verification
AC-01	A visitor can submit a valid domain query and receive a normalized response.	API integration test + browser test.
AC-02	Invalid input is rejected server-side with a safe, useful response.	Validation unit/API tests.
AC-03	Every simulated result is visibly labelled; no simulated result claims real availability.	API assertions + UI review.
AC-04	A user can register, log in, and log out.	API tests + E2E test.
AC-05	A user cannot access another user's domain or order by changing an ID.	Authorization integration tests.
AC-06	A demo order uses server-calculated prices and explicit demo status.	Service/API tests.
AC-07	Repeated confirmation cannot create duplicate effects.	Idempotency/state-transition test.
AC-08	The application provides loading, empty, success, and failure states for the main flow.	Manual and browser tests.
AC-09	No secrets, real personal data, or payment data are present in the repository.	Repository review and secret scan.
AC-10	A clean checkout can be installed, migrated, tested, and run using documented instructions.	Fresh-environment verification.
AC-11	API failures do not expose stack traces or sensitive internals to clients.	Error-path tests.
AC-12	Primary pages are keyboard operable and usable on mobile widths.	Accessibility and responsive review.

19. Risks, assumptions, and dependencies

Risk / assumption	Impact	Mitigation
No registrar API credentials	Live availability and registration cannot be demonstrated.	Use an explicit mock provider; document integration boundary and avoid false claims.
No payment sandbox access	Payment integration cannot be verified.	Complete order workflow without real payments; optionally add sandbox later.
Scope expansion	Core project may remain unfinished.	Enforce MVP scope and milestone acceptance criteria.
Security implementation errors	Account data could be exposed.	Use vetted libraries, threat-model ownership checks, and test negative cases.
External provider instability	Search or checkout could fail.	Timeouts, safe error mapping, mocks in tests, and clear UI messaging.
Deployment configuration differences	App works locally but not online.	Validate env vars, document build/run steps, test staging deployment.
Pricing confusion	Users may mistake demo values for real offers.	Label indicative values and use explicit currency/term assumptions.
Time constraints	Optional features reduce quality of essentials.	Prioritize P0, defer P1/P2 until core flow is complete.

19.1 Assumptions

- The developer has working knowledge of JavaScript and can implement full-stack features.
- Node.js, Git, and PostgreSQL are installed locally.
- The initial implementation uses React and Express with JavaScript ES modules.
- The product is a portfolio demonstration, not a commercial registrar at launch.
- The initial user base is small; the design should be sound without premature distributed-system complexity.

19.2 Dependencies

- A compatible Node.js LTS release and package manager.
- PostgreSQL instance and database credentials.
- Stable maintained packages for authentication, validation, ORM, and tests.
- Optional registrar/payment provider account and sandbox credentials for later integration.
- Hosting provider supporting the chosen frontend, API, and database deployment model.

20. Milestones and delivery plan

Milestone	Deliverables	Exit criteria
M0 — Requirements and repository	PRD, Git repo, README skeleton, agreed architecture.	Scope and initial structure are committed.
M1 — API foundation	Express app, health route, config validation, error handling, test setup.	Health route and initial tests pass.
M2 — Domain search	Validation, provider adapter, search API, frontend search/results.	Invalid input and simulation semantics tested end-to-end.
M3 — Persistence	PostgreSQL, ORM, migrations, seed strategy.	Fresh database can migrate and run tests.
M4 — Authentication	Registration, login/logout, session security, protected routes.	Authentication and cross-user authorization tests pass.
M5 — Dashboard and records	Domain record views and allowed demo edits.	Ownership and validation tests pass.
M6 — Demo order workflow	Server-calculated pricing, order state machine, demo confirmation.	No real charge; duplicate effects prevented.
M7 — Quality and security	E2E tests, accessibility pass, dependency checks, logging.	P0 acceptance criteria pass.
M8 — Publish and deploy	Public repo, screenshots, architecture docs, live demo if feasible.	Reviewer can reproduce or try the core flow.

The schedule should be estimated after M0 based on available time and any setup issues. Each milestone is complete only when its exit criteria are met.

21. GitHub repository and documentation requirements

21.1 Repository checklist

- Descriptive repository name and one-line description.
- README with project purpose, screenshots, features, stack, architecture, setup, tests, environment variables, API docs, and limitations.
- `.gitignore` excludes node_modules, environment secrets, build output, and test artifacts.
- `.env.example` contains placeholders only.
- Lockfiles are committed for reproducible installation.
- Meaningful commits that reflect actual work.
- Issues or a project board for tracked tasks if useful.
- No copied proprietary code, leaked credentials, real user data, or misleading claims.

21.2 Suggested README sections

1. Project overview and demo disclaimer
2. Features and screenshots
3. Architecture diagram

4. Technology stack
5. Prerequisites
6. Local installation and environment setup
7. Database creation, migration, and seed commands
8. How to run frontend and backend
9. How to run tests
10. API endpoint summary and error format
11. Security notes and known limitations
12. Deployment instructions
13. Roadmap and contribution notes

21.3 Portfolio presentation

The project description should accurately state which features are simulated and which are live. Screenshots should use synthetic data. If a public demo is deployed, provide demo credentials only if the environment is intentionally configured for that purpose, and never expose production accounts or privileged access.

22. Open decisions and glossary

22.1 Decisions to confirm before implementation

Decision	Default recommendation
Frontend build tool	Vite + React.
ORM	Prisma, with migrations committed.
Authentication	Server-side sessions in secure HttpOnly cookies; CSRF protection for state-changing requests.
Domain provider	Mock adapter for MVP; real provider only when access and terms permit.
Payments	No real payments in MVP; optional test-mode integration later.
Currency and demo prices	Use explicit NGN pricing assumptions and integer minor units; label all values as illustrative.
Deployment	Choose after core app works; frontend, API, and PostgreSQL hosting must be compatible.
Styling	Tailwind CSS or a similarly consistent styling system.

22.2 Glossary

Term	Meaning
Registrar	A service that enables domain registration through registry/registrar systems.
Registry	The operator responsible for a top-level domain's authoritative registration database.
DNS	The Domain Name System; resolves names to records and services.
RDAP / WHOIS	Protocols/services that expose registration-related data; neither alone guarantees purchase availability.
Provider adapter	An application boundary that normalizes interactions with an external service.
Idempotency	A property that allows a repeated request to avoid repeating the same effect.
CSRF	Cross-Site Request Forgery, where a browser is induced to make an unwanted authenticated request.
CORS	Browser policy controlling cross-origin requests; it is not a substitute for authentication.
MVP	Minimum viable product: the smallest coherent version that meets the core requirements.
Acceptance criterion	A testable condition that must be satisfied for a feature or milestone to be accepted.

End of document. Treat this PRD as the baseline for implementation; update it when a deliberate scope or architecture decision changes.